02 / ALGEBRA / EX25

Guía del ejercicio EX25

Parte A: álgebra pura · Parte B: fuerzas como vectores.

DOCUMENTO 03 / EJERCICIO

Qué aprenderás

- Comprobar manualmente operaciones algebraicas con vectores.
- Representar fuerzas mediante unitarios y componentes.
- Construir tu propio script con NumPy y verificar su salida.

Requisitos previos: Guías de teoría y referencia EX25. No se requieren JSON, pytest ni estructuras de proyectos complejas.

Orden de lectura: 1. Teoría → 2. Referencia → 3. Ejercicio → 4. Pseudocódigo → 5. Programación.

1. Situación de partida y objetivo

Trabajaremos con **dos actividades en el mismo EX25**. La parte A permite confirmar que entendemos las operaciones vectoriales sin interpretar aún un sistema físico. La parte B emplea esas mismas operaciones para sumar dos fuerzas. Es recomendable resolver primero en papel y después reproducir los cálculos en Python.

2. Parte A — Ejercicio algebraico resuelto

Datos: vectores sin unidades.

$$\mathbf{u} = (3, -1), \qquad \mathbf{v} = (-2, 4)$$

A1. Suma:

$$\mathbf{u} + \mathbf{v} = (3 - 2, \ -1 + 4)$$

$$\mathbf{u} + \mathbf{v} = (1, 3)$$

A2. Combinación lineal:

$$2\mathbf{u} - \mathbf{v} = (6, -2) - (-2, 4)$$

$$2\mathbf{u} - \mathbf{v} = (8, -6)$$

A3. Normas:

$$\|\mathbf{u} + \mathbf{v}\|_2 = \sqrt{10} \approx 3.1623$$

$$\|2\mathbf{u} - \mathbf{v}\|_2 = \sqrt{100} = 10$$

Tu práctica: programa estos cálculos y comprueba que coinciden con los resultados manuales. Después repite por tu cuenta con los vectores adicionales:

$$\mathbf{a} = (2, -1), \qquad \mathbf{b} = (-3, 5)$$

Obtén **a + b**, **2a - b** y ambas normas. Valores de control:

$$\mathbf{a} + \mathbf{b} = (-1, 4), \qquad 2\mathbf{a} - \mathbf{b} = (7, -7)$$

$$\|\mathbf{a} + \mathbf{b}\|_2 = \sqrt{17} \approx 4.1231$$

$$\|2\mathbf{a} - \mathbf{b}\|_2 = \sqrt{98} \approx 9.8995$$

3. Parte B — Problema de ingeniería: fuerzas 2D

Dos fuerzas actúan sobre un punto material en un plano horizontal-vertical. Elegimos un sistema cartesiano **x-y**. Sus componentes se expresan mediante los vectores unitarios de los ejes:

$$\hat{\mathbf{i}} = (1, 0), \qquad \hat{\mathbf{j}} = (0, 1)$$

Fuerzas dadas:

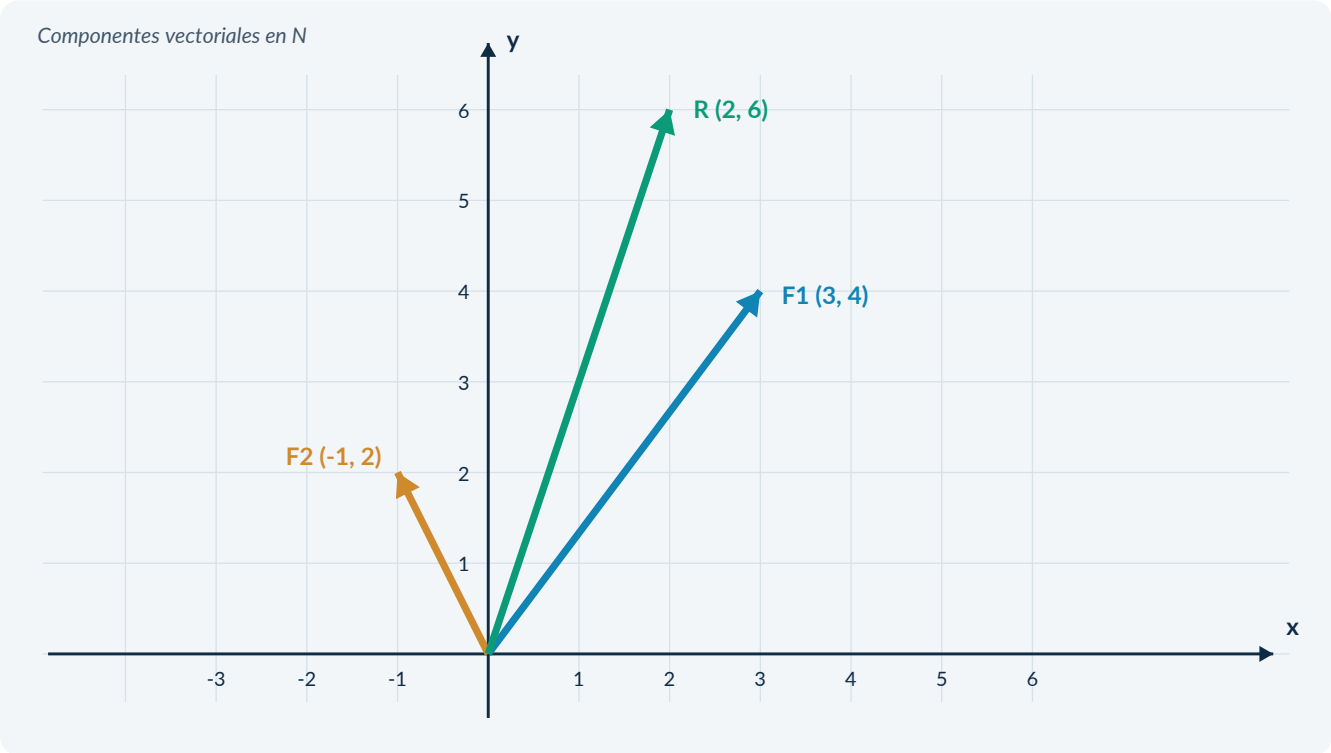
$$\mathbf{F}_1 = (3\hat{\mathbf{i}} + 4\hat{\mathbf{j}})\ \mathrm{N}$$

$$\mathbf{F}_2 = (-1\hat{\mathbf{i}} + 2\hat{\mathbf{j}})\ \mathrm{N}$$

Las letras **î** y **ĵ** con sombrero indican **direcciones unitarias**, no números complejos. Los coeficientes 3, 4, -1 y 2 son las componentes numéricas de las fuerzas en newtons.

La misma información, en forma de pares ordenados, es:

$$\mathbf{F}_1 = (3, 4)\ \mathrm{N}, \qquad \mathbf{F}_2 = (-1, 2)\ \mathrm{N}$$



4. Qué tienes que calcular

- Construye los vectores **F1** y **F2** como arrays de NumPy.
- Calcula por separado la magnitud de cada fuerza.
- Calcula el vector resultante **R = F1 + F2**.
- Calcula la magnitud de la resultante.
- Imprime los resultados con unidades y al menos tres decimales.
- Comprueba los resultados a mano antes de dar el programa por correcto.

5. Datos de control (no son el código solución)

Magnitud	Resultado esperado
Componentes de F1	(3, 4) N
Componentes de F2	(-1, 2) N
Norma de F1	5.000 N
Norma de F2	2.236 N (aprox.)
Vector resultante	(2, 6) N
Magnitud resultante	6.325 N (aprox.)

6. Requisitos de la implementación

Ubicación del ejercicio y estructura de carpetas:

```
Python_Engineering/
└── 02_numerical_methods/
    └── algebra/
        └── class_17_vectors/
            ├── EX25_vectores_y_fuerzas.py
            └── docs/
                ├── ES/
                └── EN/
```

Archivo a programar: `EX25_vectores_y_fuerzas.py`. Por ahora basta con **un único script**. La carpeta **docs** contiene las guías en español (ES) e inglés (EN). No necesitas archivos de datos, carpetas src, JSON ni tests.

El programa debe funcionar ejecutándolo desde el terminal de VS Code con `python EX25_vectores_y_fuerzas.py`. Puedes separar la parte A y la B mediante comentarios descriptivos.

7. Orientación para diseñar el pseudocódigo

Antes de abrir el editor, escribe en papel el orden lógico del programa: definir datos → transformarlos en arrays → ejecutar operaciones → calcular normas → imprimir resultados → contrastar con el ejemplo resuelto. La **Guía de pseudocódigo** desarrolla ese razonamiento y los puntos de control.

8. Pistas de programación (colocadas al final)

Las siguientes líneas muestran únicamente la sintaxis que necesitarás. **No constituyen la solución completa**; tendrás que decidir cómo construir el programa.

```
import numpy as np

example = np.array([6.0, 8.0])
length = np.linalg.norm(example)
print(f"Example magnitude: {length:.3f}")
```

Recuerda: los operadores +, - y * actúan componente a componente cuando los operandos son arrays NumPy. Puedes consultar la sintaxis de cada uno en la guía de referencia.